

BIC Talent Interview Guide for Research Engineers

This document provides a comprehensive interview framework and set of recommendations for roles within the SWE discipline, focusing on Senior and Principal Research Engineer positions. It outlines role expectations, collaboration models, required technical skills, tools, coding proficiency, evaluation criteria, and behavioral competencies to help interviewers assess candidates effectively.

BIC Talent Interview Guide for Research Engineers	1
Role Expectations.....	2
Assessment Process Overview	4
Assessment Flow for Candidates	5
Alignment with Industry Standards and Competitive Differentiation.....	7
Loop Construction Samples	8
Question Bank.....	9
ML/AI Depth & Breadth.....	9
Coding.....	13
Evaluation Rubric	15
Algorithm and Coding Evaluation Rubric.....	15
ML/AI Depth & Breadth Evaluation Rubric.....	16
Behavioral Profession Specific Questions.....	16
Key points to remember	19

Role Expectations

Discipline	Senior Research Engineer IC4 – L63/64 Senior Research Engineer (IC4 L6364).pdf	Principal Research Engineer IC5 – L65–66 Principal Research Engineer (IC5 L6566).pdf	Senior Principal Research Engineer IC6 – L67 Senior Research Engineer (IC4 L6364).pdf
What this role does:	The Senior Research Engineer role drives the development and integration of advanced AI solutions within BIC. This position bridges the gap between state-of-the-art research and customer-facing features, driving systems-level innovation across models, infrastructure, and deployment. It also champions responsible AI by embedding fairness, safety, privacy, and performance throughout the development process to deliver inclusive, ethical, and production-grade AI systems.	The Principal Research Engineer role sets the technical vision and leads transformative AI initiatives that shape Microsoft’s products and services. Positioned at the intersection of research, engineering, and product strategy, this role defines and executes technical strategies for foundational models, multi-agent systems, and next-generation Copilot experiences within BIC. It drives large-scale innovation, guides cross-team delivery of scalable and responsible AI systems, and translates research breakthroughs into meaningful customer and business impact while providing technical leadership and mentorship across the organization.	The Senior Principal Research Engineer at Microsoft defines the technical vision and leads high-impact AI initiatives that shape Microsoft’s products and services. The role bridges advanced research, engineering, and product strategy to deliver scalable, real-world solutions. As a recognized technical leader, this person guides cross-organizational strategy, mentors senior engineers, and represents Microsoft within the global research community.
Collaboration & Mentorship	Comfortable to drive and effectively participate in feature level collaboration. Can mentor and is sought after by early career REs	Comfortable to drive and effectively lead feature area or product level collaboration. Can mentor and is sought after by IC4 or early career REs and Research Scientists	Comfortable to drive and effectively lead product level or cross product group collaboration. Can mentor and is sought after by IC4/5 REs and Research Scientists

Communication & Impact	Effective and confident feature/squad level cross disciplinary communications.	Effective and confident feature/squad/product level cross disciplinary communications.	Effective and confident cross team and executive communications (CVP and above)
Develops production code:	Experiences in developing and debugging code in production highly desired – including model inference code, MLOps pipeline coding, and deploy models via CI/CD pipelines.	Experiences in developing, debugging, and monitoring code in production expected - including model inference code, and MLOps pipeline coding and ML system optimization	Extensive experiences in developing, debugging, and monitoring code in production required - including model inference code, MLOps pipeline coding, ML system optimization, and end-to-end model lifecycle
Experimentation & Evaluation	Experiences in designing and implementing some eval components, including datasets, metrics, offline or online experimentations highly desired	Experiences in designing and implementing end to end evals, including datasets, metrics, offline and online experimentations highly expected	Extensive experiences in designing and implementing end-to-end evals, including datasets, metrics, offline and online experimentations required. Drives scientific rigor into the engineering process
ML/AI Theory	College/postgraduate level course work or equivalent in basica applied math subjects (e.g., linear algebra, probability/statistics), solid understanding of common classical ML and deep learning algorithms	+ possess a proven track record of applying and deploying optimization, statistical learning, and reinforcement learning techniques within industrial or academic environments.	+ practical experience in implementing reinforcement learning at scale, a solid understanding of causal inference, LLM foundation model, self-supervised learning, responsible AI and alignment techniques.
Internal Tools Used	N/A	N/A	N/A
Usage of Third-Party Tools	AI assisted IDE, SWE agent, Eval tools (e.g., Braintrust, Statsig), LLM Application Development/Orchestration Framework (e.g., LangChain), CD/CI for ML	AI assisted IDE, SWE agent, Eval tools (e.g., Braintrust, Statsig), LLM Application Development/Orchestration Framework (e.g., LangChain), CD/CI for ML	AI assisted IDE, SWE agent, Eval tools, LLM Application Development/Orchestration Framework (e.g., LangChain), CD/CI for ML
Coding or Query Languages	Python, CUDA, SQL	Python, C++, CUDA, SQL, Spark/PySpark, R, Graph Query Languages	Python, C++, CUDA, Triton, SQL, Spark/PySpark, R, Graph Query Languages

Coding Expectations	Debug, and deploy in production	Design, debug, and monitor in production	Design, rapid prototype, debug, and monitor in production
LLM & Generative AI	Some academic or industry experiences in developing or deploying fine-tuned AI systems based on leading close or open-source foundation models highly desired	Strong industry experiences in developing or deploying fine-tuned AI systems (especially supervised and reinforcement fine-tuning) based on leading close or open-source foundation models expected	Extensive industry experiences in developing and deploying fine-tuned AI systems (especially supervised and reinforcement fine-tuning) based on leading close or open-source foundation models desired
Deployment & MLOps	Experiences desired	Extensive experiences expected	Extensive and end-to-end experiences required
Ethics & Responsible AI	Ensures ethical deployment and compliance in production systems	Ensures ethical deployment and compliance in production systems	Ensures ethical design, deployment and compliance in production systems
Minimum Qualification	Advanced degree (Bachelor's, Master's, or PhD) in a technical field with 2–4+ years of experience in applied ML, AI research, or product engineering, including work with generative AI or LLMs. Proficiency in Python and deep learning frameworks (e.g., PyTorch, JAX, TensorFlow), with hands-on experience deploying, shipping, and maintaining fine-tuned or multimodal AI models in production.	Advanced degree (Bachelor's, Master's, or PhD) in a technical field with 5–10+ years of experience in applied AI/ML research and product engineering, supported by a strong publication record. Demonstrated leadership in delivering large-scale AI systems, mentoring senior engineers, and shaping product direction, with expertise in Python, deep learning frameworks (e.g., PyTorch, TensorFlow), and modern MLOps stacks.	Advanced degree (Bachelor's, Master's, or PhD) in a technical field with 10–15+ years of experience in applied AI/ML research and product engineering, supported by a strong publication record. Demonstrated leadership in designing and delivering large-scale AI systems, mentoring senior engineers, and shaping product direction, with expertise in Python, C++, deep learning frameworks (e.g., PyTorch, TensorFlow), and modern MLOps stacks.

Assessment Process Overview

The assessment process described below is designed for evaluating candidates applying for Research Engineering roles at levels IC 63–67, which correspond to Career Stages IC4, IC5, and IC6.

The main goal is to provide a clear and consistent foundation for the BIC Hiring Committee, ensuring alignment on evaluation criteria across all applicants. This approach supports a shift toward skill-based hiring practices, utilizing standardized interview loops that promote fairness and objectivity throughout the candidate assessment process.

Assessment Flow for Candidates

Both internal and external candidates go through a defined evaluation flow, varying slightly depending on the career stage. The process includes an initial screening followed by a series of full interviews.

Initial Screening

Career Stage	Initial Screening Type	Duration	Assessment Criteria
IC4 L63 & L64	Online Technical Screening (OTS via HackerRank)	45 minutes	Core coding question (LeetCode style) and ML coding question
IC5 L65 & L66	Phone Screen	60 minutes	Live coding question (LeetCode style) and ML fundamentals
IC6 L67	Phone Screen	60 minutes	Live coding questions (LeetCode style) ML fundamentals System/architecture design thinking

Full Interview Series

After the initial screening, candidates proceed to four full interviews, each focused on distinct skill areas.

Career Stage	Interview 1	Interview 2	Interview 3	Interview 4 (AA)

IC4 L63 & L64	Coding + Customer Focus	ML System Design and Collaboration	ML Depth and Breadth + Drive for Results	Problem Solving Case Study + Growth Mindset + Innovation
IC5 L65 & L66	Coding + Customer Focus	ML System Design and Collaboration	ML Depth and Breadth + Influencing for Impact	Problem Solving Case Study + Growth Mindset + Innovation
IC6 L67	Coding + Customer Focus	Product sense ML System & Architecture Design	ML/Deep learning Depth and Breadth LLM fundamentals Collaboration and Influencing for Impact	Problem Solving Case Study Executive communications Growth Mindset + Innovation

Interview Focus Areas

Coding: candidates will be evaluated on their coding ability (algorithms and, typically in Python) and ML frameworks.

ML System and Architecture design: Candidates are asked to architect and design scalable machine learning systems for large-scale enterprise use cases. The interview can *also* emphasize designing systems for scalability and cutting-edge research, often prioritizing efficiency, distributed training, and novel architecture. The interview should also probe if and how the candidates make design choices, and trade-offs. This design interview will lean toward algorithmic scalability and innovation, not necessarily customer-facing enterprise requirements. Based on the role needed, we can alternate.

ML/Deep learning depth and breadth: Assessment covers model development (including dataset strategy) and frameworks (supervised, unsupervised, deep learning), optimization techniques, and evaluation metrics.

Problem solving case study: Candidate solves a real-world problem with responsible AI mindset (e.g., bias mitigation in a recommendation system). This segment evaluates the candidate's ability to innovate and demonstrate a growth mindset during brainstorming and solution development.

Requisition Profession and Discipline (from MSFT Role Library): Software Engineer

Although it is machine learning, we are still assessing their coding and system/architecture design (ML design though). The Research Engineers will work on prototype to production.

Alignment with Industry Standards and Competitive Differentiation

Based on findings from publicly available information, the breakdown below illustrates how the proposed assessment criteria align with prevailing industry standards. This comparison focuses specifically on direct competitors, including OpenAI, Anthropic, and Google.

Industry Alignment

The proposed model closely reflects the assessment focus areas emphasized by leading organizations in the field. Key criteria such as coding proficiency, machine learning system design, depth and breadth of ML expertise, and problem-solving abilities are standard components of interviews at OpenAI, Anthropic, and Google. This alignment ensures that our evaluation process remains relevant and consistent with those of top-tier competitors.

Unique Attributes of the Proposed Model

While the model is grounded in established industry practices, it also incorporates distinctive elements tailored to Microsoft's product requirements and cultural values. These unique features set our approach apart, providing additional relevance and differentiation that support our organizational goals and identity.

Dimension	The above proposed Model	Google/OpenAI/Anthropic
Coding	Strong focus (IC4-6)	Strong focus
ML System and Architecture Design	Enterprise-scale use cases	Scalable systems, research-oriented
ML Depth & Breadth	Explicit in IC4-6	Explicit in OpenAI & Anthropic

Dimension	The above proposed Model	Google/OpenAI/Anthropic
Responsible AI	Case study	Safety & ethics integrated
Communications Effectiveness	Strong focus (IC6)	Strong focus
Customer Focus	Unique differentiator	Rarely emphasized
Behavioral Competencies	Growth mindset, innovation, influencing impact	Collaboration & adaptability (less structured)

Loop Construction Samples

Senior Research Engineer Loop Structure	Principal & Sr Research Engineer Loop Structure
Coding: Prototyping experimentation	Leadership (Model, Coach, Care) Defining and leading long-term research agendas
ML/AI Depth& Breadth: Using Questions Bank below to assess Depth and Breadth pick two of the following applicable to Research Science: • Case Study: Optional — more relevant if tied to applied research or product collaboration • Experiment Design: Assess ability to design scientific experiments and validate hypotheses • Architecture Walkthrough: Optional — useful if research involves system design or tool development • Research Critique: Essential — assess ability to critique and extend state-of-the-art research	ML/AI Depth& Breadth: Using Questions Bank below to assess Depth and Breadth pick two of the following applicable to Research Science: • Case Study: Optional — useful if managing applied research or product-facing collaborations • Experiment Design: Evaluate ability to support rigorous scientific experimentation • Architecture Walkthrough: Optional — relevant if managing research infrastructure or tooling • Research Critique: Essential — assess ability to evaluate and guide research direction • Ethics Scenario: Leads ethical research practices and policy development
Focus driven towards understanding: • Scientific impact and publication record • Collaboration with academia and internal teams • Ethical leadership in research practices	Focus driven towards understanding: • Scientific impact and publication record • Collaboration with academia and internal teams • Ethical leadership in research practices

	For Senior Research Engineer also use Technical Retrospective Interviews L67 Research Eng.pdf as a guide.
• XFN Collaboration • Culture- Growth Mindset, One Microsoft • Customer Obsession	• XFN Collaboration • Culture- Growth Mindset, One Microsoft • Customer Obsession

Question Bank

ML/AI Depth & Breadth

Question	Expected Answer/Look for
Category: LLM Optimization	
Describe a time you used distillation or quantization. What were the tradeoffs?	Practical experience with model compression, understanding of latency vs. accuracy tradeoffs.
How do you fine-tune a pre-trained LLM for a domain-specific task?	Knowledge of transfer learning, LoRA, prompt tuning, and evaluation metrics.
What are the challenges in aligning LLMs with human intent?	Familiarity with RLHF, prompt engineering, hallucination mitigation, and safety.
How do you evaluate the performance of a generative model?	Use of BLEU, ROUGE, perplexity, human evaluations, and task-specific metrics.
What techniques would you use to reduce inference latency in large models?	Model pruning, quantization, caching, and efficient transformer variants like Linformer.
How would you modify a Transformer-based LLM to reduce inference latency when deploying on low-resource devices, while minimizing performance loss?	-Proposes compression techniques like quantization, pruning, and knowledge distillation. - Mentions architectural changes such as using lightweight models and attention optimizations. - Refers to deployment optimizations like key-value caching, batch inference, and efficient runtimes.
You're fine-tuning an LLM using Reinforcement Learning from Human Feedback (RLHF). What challenges might arise with the reward model, and how would you address potential reward hacking?	-Defines reward hacking as the model exploiting flaws in the reward signal. - Identifies reward model issues such as proxy metrics, overfitting, and rater bias. - Proposes mitigations like ensembling, uncertainty penalties, adversarial testing, and human audits.

Category : ML/ LLM Systems Design	
How do you monitor deployed ML models in production?	Drift detection, shadow deployment, A/B testing, and logging.
What tools do you use for model training and experiment tracking?	MLflow, Weights & Biases, Azure ML, or internal platforms.
Describe your experience with distributed training.	Use of Horovod, DeepSpeed, or custom data/model parallelism strategies (only candidates for L67 roles who have applied scientist background).
How would you design a scalable ML pipeline for real-time inference?	Use of streaming data, model serving frameworks, and autoscaling.
How do you ensure reproducibility in ML experiments?	Version control, containerization, and consistent data splits.
How would your overall design change if you switched from a small classification model to using an LLM	Awareness of latency, cost, infra changes, interpretability, and throughput implications
In what scenarios would you still prefer training a classical classifier over using an LLM prompt?	Reasoning around cost, speed, deployment footprint, explainability, edge use-cases, hallucination cost, user feedback
Would you fine-tune an LLM to replicate the classical model's performance? Why or why not?	Recognition that some use-cases don't justify LLM fine-tuning; may recommend using prompts or adapters instead
What risks do you foresee in blindly replacing a classical classifier with an LLM prompt?	Non-determinism, interpretability, debuggability of failure cases, mitigating customer-reported errors
Present an existing classical pipeline and ask candidates to point at specific components they would: Keep intact, augment with LLMs, fully replace and why.	Filter out candidates who only think in end-to-end LLM terms vs. those who can strategically redesign or augment systems in modular, pragmatic ways.
Given a practical scenario, how to build a model to support that scenario? A few examples: • how to design and build a model for customer support chatbot? • how to build a LLM-based code assistant? • how to build a model for meeting summarization?	It would need candidate to start from task framing, training data collection, model training, evaluation, go to prod and iterations Candidate needs to • clarify requirement to get clean task framing • design training data curation strategies, data cleaning, synthetic data generation etc. • design evaluation mechanism • choose the proper training technique to train a model • how to do A/B testing • how to get feedback and iterate

Category: GenAI Infrastructure	
What are the key components of a Transformer model?	Understanding of self-attention, positional encoding, encoder-decoder structure.
How do you manage GPU memory efficiently during training?	Gradient checkpointing, mixed precision training, and batch size tuning.
What infrastructure considerations are important for training large generative models?	Distributed training, data throughput, and fault tolerance.
How do you handle model versioning and rollback in GenAI systems?	Model registries, CI/CD pipelines, and canary deployments.
What are the challenges in scaling GenAI workloads across clusters?	Orchestration, load balancing, and inter-node communication.
Category: Pre-training and Post-training LLM Models	
What data curation strategies do you use for pre-training LLMs?	Deduplication, filtering, and domain balancing.
You're training a 70-billion-parameter Transformer-based LLM and observe intermittent large spikes in the training loss, especially during later training stages. What could be causing this behavior, and how would you diagnose and address it?	Identifies common causes such as high learning rate, precision instability (e.g., FP16 overflow), outlier batches, or optimizer state drift. - Diagnoses via gradient norm tracking, logging per-batch loss and input statistics, and visualizing activation values. - Suggests fixes such as gradient clipping, learning rate schedules, stable optimizers, and improved mixed-precision handling.
How do you evaluate the quality of pre-training data?	Perplexity, diversity metrics, and manual inspection.
What is your approach to post-training alignment?	RLHF, supervised fine-tuning, and safety tuning.
How do you detect and mitigate hallucinations in LLM outputs?	Fact-checking, retrieval augmentation, and prompt engineering.
What are the tradeoffs between pre-training from scratch vs. fine-tuning?	Cost, data availability, and performance considerations.
In your view, why is reinforcement learning more critical than supervised fine-tuning (SFT) for ensuring safety in LLMs or agentic systems?	Clear articulation of the limitations of SFT (e.g., static labels, lack of feedback loops). Strategic framing of RLHF (Reinforcement Learning from Human Feedback) as a dynamic alignment mechanism. Awareness of safety frameworks (e.g., reward modeling, adversarial testing, red teaming).

What reinforcement learning algorithms have you championed or scaled in production, and how did they influence product or platform outcomes?	Strategic selection of RL algorithms (e.g., PPO, DDPG, SAC) based on business or platform constraints. Experience leading teams that deployed RL at scale (e.g., for ranking, personalization, agentic behavior) Discussion of trade-offs (e.g., sample efficiency, stability, interpretability). Evidence of measurable impact (e.g., latency reduction, engagement lift, safety improvements).
How do you scale safety techniques—like reward modeling, red teaming, or interpretability—across different research areas or product verticals?	A systems-level view of safety as a platform capability, not a one-off fix. Experience building reusable safety infrastructure (e.g., evaluation harnesses, feedback loops, dashboards).
Category: General Infra Applied Science	
How do you choose between different ML models for a given task?	Model complexity, interpretability, data size, and performance tradeoffs.
Describe how you would handle imbalanced datasets.	SMOTE, class weighting, stratified sampling, or anomaly detection.
How do you ensure safety and fairness in ML systems?	Bias detection, adversarial testing, and fairness metrics.
What are the challenges of training LLMs on shared infrastructure?	GPU scheduling, memory constraints, checkpointing, distributed training (e.g., DeepSpeed, FSDP), and fault tolerance
How would you design a scalable feature store for multiple ML teams?	Concepts like offline/online store separation, consistency, latency, and integration with training/inference pipelines.
What is your approach to data synthesis for model improvement?	Synthetic data generation, augmentation, and filtering strategies.
How do you handle data versioning and lineage in a multi-tenant AI platform?	Use of metadata stores, lineage tracking tools (e.g., Amundsen, DataHub), and governance practices.
How do you collaborate with engineering teams to deploy models?	Experience with APIs, model packaging, and cross-functional communication.
Category: General Infra Research Science	
What's a recent ML/GenAI paper that inspired you, and why?	Ability to synthesize research and relate it to practical applications.
How do you validate research hypotheses in production settings?	A/B testing, offline evaluation, and user feedback.

Category: General Data Science	
How would you optimize a data pipeline for large-scale model training?	Familiarity with distributed data processing (e.g., Spark, Dask), data sharding, caching strategies, and I/O bottlenecks.
Explain how you would monitor model drift in a deployed system.	Use of statistical tests (e.g., KL divergence), data distribution checks, and retraining triggers.
How do you ensure reproducibility in ML experiments?	Use of version control (e.g., DVC), containerization (Docker), seed setting, and experiment tracking tools (e.g., MLflow, W&B).

Coding

Production and ML/AI Integration Coding Questions(Conduct using shared coding Environment)-	
Design a logging system for a production ML pipeline that supports multiple models and tracks inference latency and errors.	Use structured logging (e.g., JSON logs), include model name/version, timestamps, latency metrics, and error codes. Store logs in a centralized system (e.g., Azure Monitor, ELK stack).
Given a trained model, how would you deploy it to serve real-time predictions with low latency and high availability?	Use a model server (e.g., FastAPI, TorchServe), containerize with Docker, deploy via Kubernetes or Azure ML endpoints, implement autoscaling and health checks.
How would you handle versioning and rollback for models in production?	Use model registry (e.g., MLflow), tag versions, deploy behind a feature flag or shadow mode, monitor performance, and rollback if metrics degrade.
Write a function that takes a stream of predictions and computes a rolling AUC score every 1000 predictions.	Maintain a buffer of last 1000 predictions and labels, use sklearn.metrics.roc_auc_score, update buffer with new entries, compute AUC periodically.
How would you integrate a model into an existing microservice architecture with minimal disruption?	Expose model as a REST/gRPC endpoint, use API gateway for routing, ensure backward compatibility, test with canary deployment.
Design a tool-using agent loop that can dynamically select between multiple tools (e.g., calculator, search, summarizer) based on user intent.	Modular design with intent classification (e.g., classifier or prompt router), memory/state management, tool abstraction, fallback/error handling. Bonus: LangChain or custom orchestration logic.
Implement a memory module for an LLM agent that stores and retrieves relevant past interactions using vector similarity.	Use of embedding models (e.g., OpenAI, HuggingFace), vector store (e.g., FAISS, Pinecone), cosine similarity, pruning/refreshing logic.
Write a caching layer for LLM inference that reduces latency and cost for repeated queries.	Hash-based or semantic caching, Redis or in-memory store, TTL, cache invalidation strategies. Bonus: prompt normalization.

Build a streaming inference pipeline for an LLM that supports token-by-token generation and client-side rendering.	Async generators, WebSockets or SSE, token buffering, integration with streaming APIs (e.g., OpenAI, vLLM).
Design a scalable architecture to serve multiple LLMs (e.g., GPT-4, Claude, Mistral) behind a unified API with routing logic.	API gateway, model registry, routing by task/cost/performance, observability (e.g., Prometheus), fallback logic.
Write a function that monitors input drift for an LLM agent and triggers retraining or alerts.	Statistical tests (e.g., KL divergence, PSI), reference distributions, logging pipelines, alerting (e.g., Azure Monitor, Prometheus).
Design a system to support multi-agent collaboration (e.g., planner + executor + verifier) for complex tasks.	Agent roles, communication protocol (e.g., message passing), task decomposition, coordination logic. Bonus: CrewAI or AutoGen.
Coding/ SQL (Conduct using shared coding Environment)-Data Science	
Data Science: SQL: ex: Which school has the highest average test score?	Proper use of aggregation, joins, and ordering.
Data Science: Python: ex Write a function to detect outliers in a dataset.	Understanding of IQR, edge case handling, and clean code.
Data Reprocessing: ex Write a function to normalize a dataset column using z-score normalization.	Understanding of mean, standard deviation, and vectorized operations.
Distributed Data Handling: ex: How would you process a 1TB CSV file to compute aggregate statistics?	Mention of chunking, Dask, Spark, or streaming approaches.
Embedding Similarity: ex : Given a matrix of sentence embeddings, find the top-3 most similar sentences for each row.	Use of cosine similarity, matrix operations, and efficient indexing.
LLM Prompt Evaluation: Ex: Given a list of prompts and LLM responses, write a function to score them using BLEU or ROUGE	Familiarity with NLP metrics and libraries like nltk or rouge-score.
Infrastructure-Aware ML: Ex : How would you implement a caching mechanism for model inference in a high-throughput API?	Use of LRU cache, Redis, or memorization strategies.
Model Evaluation: Ex : : Implement a function to compute precision, recall, and F1-score from scratch.	Understanding of classification metrics and edge case handling.
Feature Engineering: Ex Given a time series of user activity logs, extract features like session length, time between sessions, and peak activity hour.	Use of pandas groupby, diff, resample, and datetime operations

Implement a tool-using agent loop : Ex Simulate an agent that receives a task, selects a tool (e.g., calculator, search), and executes a plan.	Planning logic, memory state, and modularity.
Design a memory module for an LLM agent : Ex Create a memory buffer that stores past interactions and retrieves relevant context using cosine similarity.	Embedding handling, vector search, and memory pruning.
Build a prompt router : Ex Given a user query, route it to the appropriate LLM or tool based on intent classification.	Use of regex, ML classifiers, or rule-based logic.

Evaluation Rubric

Algorithm and Coding Evaluation Rubric

This rubric provides guidance for evaluating candidates' algorithm and coding skills. It aligns with the expectations outlined in the interview guide and helps interviewers assess performance consistently.

Criteria	Below Expectations	Meets Expectations	Exceeds Expectations
Problem Understanding	Struggles to understand the problem or misses key constraints.	Understands the problem and constraints with minimal clarification.	Quickly grasps the problem, asks insightful clarifying questions.
Algorithm Design	Proposes an inefficient or incorrect approach.	Proposes a correct and reasonably efficient algorithm.	Proposes an optimal or innovative algorithm with clear justification.
Code Quality	Code is incomplete, poorly structured, or contains syntax errors.	Code is complete, readable, and logically structured.	Code is clean, modular, and demonstrates best practices.
Testing & Debugging	Does not test or fails to identify bugs.	Tests code with common cases and fixes issues.	Thoroughly tests edge cases and proactively debugs.
Communication	Struggles to explain thought process or code.	Explains approach and code clearly.	Communicates clearly and effectively, including trade-offs and alternatives.

ML/AI Depth & Breadth Evaluation Rubric

Applied Science in ML/AI

Below Expectations	Meets Expectations	Exceeds Expectations
Struggles to translate ML/AI concepts into scalable, production-grade systems; limited understanding of model deployment, monitoring, or real-world constraints.	Effectively applies ML/AI to real-world problems; understands system constraints, deployment, and monitoring; collaborates across engineering and data teams.	Drives end-to-end ML/AI solutions; optimizes pipeline performance and scalability; mentors others in best practices for applied ML/AI, continuous integration, and ethical considerations. Be able to provide potential alternative solution(s) if asked and compare the options contextually.

Behavioral Profession Specific Questions

Technical strength		
Competency description: Demonstrates technical depth, problem solving, Adaptability, and curiosity. Applies continuous learning to technical challenges.		
Behavioral questions - Tell me about a time when you sought out a challenging technical problem. - Tell me about a time when you mentored others in your area of technical expertise. - Tell me about a time when you had to learn a new technology. What did you do to gain expertise? - Describe a situation when you came up-to-speed quickly on new tools, methods, or processes. - Describe a situation when you were able to accelerate your own learning curve on a project or assignment. - Describe a situation when you sought out a new engineering or project/program management challenge beyond your role requirements.		
Competency Scale		
ABOVE EXPECTATIONS: Highly detailed; large scope/role	MEETS EXPECTATIONS: Adequate detail; moderate scope/role	DOES NOT MEET EXPECTATIONS: Missing detail; small scope/role
Demonstrated substantial knowledge of technical concepts, facts and principles of particular subject matter domain	- Demonstrated knowledge of technical concepts, facts, and principles of particular subject matter domain - Applied technical thoroughness and rigor to programs, products, and services	Demonstrated minimal knowledge of technical concepts, facts, and principles of particular subject matter domain

- Anticipated the need for a technology standard and led continuous improvement efforts - Identified new tools or technologies to incorporate into their work - Evangelized and taught others how to reach beyond existing solutions and technologies to incorporate new ideas	- Proactively sought out new technical challenges to improve their skills - Taught others how to reach beyond existing solutions and technologies to incorporate new ideas	- Did not apply technical thoroughness and rigor to programs, products, and services - Encountered new technical challenges but did not actively seek them out - Did not encourage others to reach beyond existing solutions and technologies to incorporate new ideas
--	---	--

Strategic thinking

Competency description :Anticipates interrelated technological, business, and cultural shifts and their relevance. Predicts disruptive future technologies. Envisions future scenarios, products, and markets, and new ways to be empowered by technology. Implements technology solutions to support the vision.

Behavioral questions

- Tell me about a time you had to balance model performance with infrastructure cost.**
- Tell me about a time when you developed a research plan to generate strategic customer or product insights.
- Tell me about a time when you actively sought out a complex computer science problem with the intention of offering an innovative solution.
- Tell me about a time when you were able to anticipate a technology shift.
- Tell me about a time when you were unable to resolve a challenging technical problem. How did you handle the situation?
- Tell me about your most successful research contribution.
- Tell me about your least successful research contribution.

Interview competency scale

ABOVE EXPECTATIONS: Highly detailed; large scope/role	MEETS EXPECTATIONS: Adequate detail; moderate scope/role	DOES NOT MEET EXPECTATIONS: Missing detail; small scope/role
Projected community and social trends to anticipate their connection to technology and product needs	Demonstrated insight into community and social trends	Did not describe any insight into community and social trends

- Identified challenging technical problems and provided creative solutions that received support from senior stakeholders - Asked appropriate questions to gain insight into challenging situations, and applied what he or she learned across domains - Developed a research agenda that is empirically derived/deeply integrated in the needs of the business	- Identified challenging technical problems and provided creative solutions - Asked appropriate questions to gain insight into challenging situations - Developed a research agenda based on rigorous analysis and insight into future needs	- Identified challenging technical problems but could not provide a solution - Asked questions to gain insight into challenging situations, but became distracted by irrelevant details - Did not develop a research agenda based on rigorous analysis and insight into future needs
--	--	--

Entrepreneurialism

Competency description Takes intelligent risks and changes the current practices. Demonstrates creativity and a willingness to take action in uncertain conditions. Tenaciously pursues new possibilities and consistently overcomes obstacles, sustaining focus on the goal. Recruits others to her or his visions and enlists their help.

Behavioral questions

- Tell me about a time when you successfully challenged the opinion of a superior or a proposed strategy.
- Tell me about a professional risk that involved a new technology or practice. What information did you gather prior to making the decision?
- Tell me about a time when you had to defend your opinion against strong opposition.
- Provide an example of an idea that you recommended that was not adopted. Why was it rejected?
- Tell me about a time when you took initiative to pursue an idea that you believed would be helpful to your organization.
- Tell me about a time when you successfully communicated your vision for a project or technology and received support from others.

Interview competency scale

ABOVE EXPECTATIONS: Highly detailed; large scope/role	MEETS EXPECTATIONS: Adequate detail; moderate scope/role	DOES NOT MEET EXPECTATIONS: Missing detail; small scope/role
Took calculated risks to achieve organizational success	Took calculated risks to achieve organizational success	Took ill-advised risks to achieve organizational success

• Challenged the status quo and offered innovative solutions to improve organizational practices • Demonstrated conviction when presenting a new idea or strategy to senior level associates • Overcame obstacles by treating them as opportunities for organizational growth	• Challenged the status quo to improve organizational practices • Demonstrated conviction when presenting a new idea or strategy to peers and direct management • Overcame obstacles to achieve goals	• Maintained the status quo and failed to improve organizational practices • Did not demonstrate conviction when presenting a new idea or strategy to peers and direct management • Was unable to overcome obstacles to achieve goals
---	---	---

Key points to remember

For any ML/AI Science / Coding Questions

Algorithm & coding should have a reasonable level of difficulty. Focus on problems that have some relationship to AI/ML but don't simply require you to call a function from SciKit Learn.

Set the expectation with the candidate that you will expect them to write the production-level code. At the end of the day the outcome of the coding interview is a written code which you will be evaluating, not pseudo code.

Raise the bar. Make sure that the coding question is appropriate for the level and position but err on the side of raising the technical bar. Candidates who are well-prepared should be able to solve your problems. We do not want to use exact LeetCode questions(!), problems should be at least on the Medium-difficulty level seen there. Note that LeetCode levels are not reliable. A LeetCode MEDIUM/HARD may assume a C++ solution (where data allocation and typing of results is complex) and may be EASY in python (or even very easy).

Try to pick problems that are not tricky but do require some insight and perhaps some auxiliary functions (avoid "if you see the solution its only one line of code!"). The solution should require at least 20 lines of real code.

Solve the problem yourself to see if the solution is challenging. For example, if the solution is just two nested loops where you append the results, then it is not hard enough.

If you are interviewing a candidate and they quickly solve the problem, be sure to have additional follow-on questions that can be added to allow the candidate to show that they are exceptional.

The goal is to produce code that is syntactically valid and works.

- Pseudo code is not the goal.
- Candidates should test their code in the time allowed.
- Utilize (shared coding platform) Hackerrank? to make the process easier and provide a consistent candidate experience
- Collect a well-formatted text copy of the solution given.
- One scheme that works well is to cut and paste into Visual Studio / VSCode, ensuring formatting, and then from VS pasting into the feedback.
- Try to read the code you pasted in. If you can't easily read it, then keep trying.

Writing Feedback

Write the feedback on the same day that the interview took place!

Use the following structure for writing feedback:

Recommendation: Hire/No Hire (there should be no weak-hire or hire with reservations. It is ok to indicate a level, however. E.g. the loop was for a L66, but the candidate performed well for a L65, not for a L67)

Summary: quick summary of the interview (1-3 sentences)

Competencies Assessment: area for non-technical assessment

Coding Question OR AI/ML Depth/Breadth Questions: full question statement and additional context like the time reserved for the question,

time the candidate took to answer, for coding: did it run

Solution Assessment: For each of the 5 areas below, write 1-3 sentences with the supporting details including a clear evaluation of performance:

[-]: below expectation

[v]: meets expectation

[+]: above expectation

Code: well-formatted code (copy to VS first!). Code works, it compiles without error.

Tests: Verbatim tests

Remember that the goal is to determine if the candidate writes code regularly and is good at it, some call this fluency. Active programming is regularly aided using online sources so perfect recall shouldn't be expected, but the candidate should have fluid recall of standard functions and not make egregious syntactical errors.

Tip: It is not uncommon to get feedback like this "I asked my typical reverse string problems. Candidate did better than average." This is not acceptable. **Keep the full problem description in a text/word document for quick copy & paste to candidate and into the feedback tool.** Also keep separated the written expectation for great vs good vs poor solutions. That can also be copy & pasted to feedback.